

ДИСЦИПЛИНА	Алгоритмы и структуры данных
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Преснецова Виктория Юрьевна, Яковлев Михаил Сергеевич, Дворецкий Артур Геннадьевич, Гиматдинов Дамир Маратович
СЕМЕСТР	2 семестр, 2024-2025 гг.

Рабочая тетрадь 2.

Сбалансированные двоичные деревья поиска

Требования
1. Язык программирования - C++. 2. Код должен быть оптимизирован для производительности и использования ресурсов. 3. Необходимо избегать избыточных вычислений и памяти. 4. Комментарии должны объяснять сложные участки кода и логику работы программы.

1. AVL-деревья

Теоретический материал
Подробные материалы о всех видах сбалансированных деревьев и операциях с ними (добавление и удаление элементов) смотрите в слайдах лекции 2 AVL-деревья AVL-дерево - сбалансированное по высоте двоичное дерево поиска: для каждой его вершины высота её двух поддеревьев различается не более чем на 1. Для

АВЛ-деревьев **сбалансированность определяется разностью высот правого и левого поддеревьев любого узла**. Если эта разность по модулю не превышает 1, то дерево считается сбалансированным. Данное условие проверяется после каждого добавления или удаления узла, и определен минимальный набор операций перестройки дерева, который приводит к восстановлению свойства сбалансированности, если оно оказалось нарушено.

Для каждого узла дерева в структуру вводится переменная – показатель баланса (разность высот правого и левого поддеревьев)

Структура данных, описывающая узел АВЛ-дерева, имеет вид:

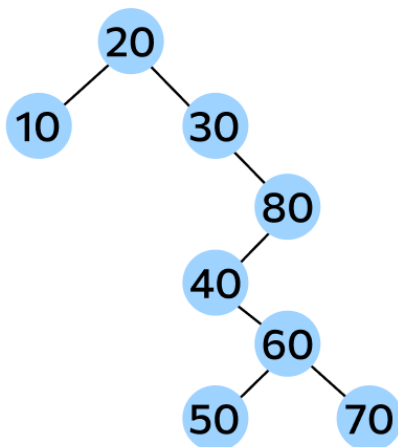
```
Struct Node{  
int key;  
Node * left;  
Node * right;  
Node * parent;  
int balans;  
};
```

Для упрощения работы алгоритмов в структуру допускается ввести указатель на родительский узел.

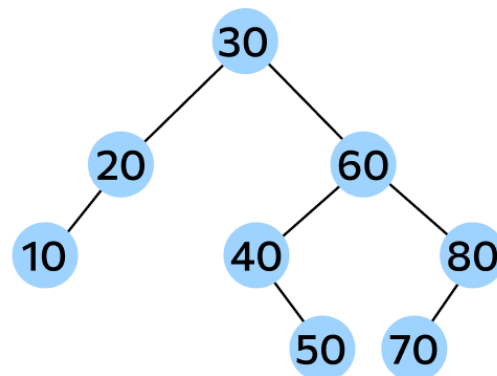
Алгоритм добавления узла:

1. Выполнить обычную вставку, как в бинарном дереве поиска.
2. Обновить высоту текущего узла.
3. Вычислить баланс-фактор.
4. Если баланс-фактор выходит за пределы $[-1, 1]$, выполнить поворот.

Бинарное дерево поиска



AVL-дерево



Отличия AVL-деревьев от бинарных деревьев поиска

Критерий	AVL-дерево (AVL)	Бинарное дерево поиска (BST)
<i>Балансировка</i>	Поддерживает баланс автоматически после каждой вставки или удаления.	Может быть несбалансированным, если элементы добавляются в порядке возрастания или убывания.
<i>Баланс-фактор</i>	Разница высот левого и правого поддеревья всегда -1, 0 или 1.	Баланс не контролируется. Разница может быть любой.
<i>Операции вставки/удаления</i>	Требует дополнительного времени на балансировку (вращения).	Вставка и удаление проще, но может привести к сильному дисбалансу.
<i>Поиск</i>	$O(\log n)$ в худшем случае.	$O(n)$ в худшем случае (при сильном дисбалансе).
<i>Повороты</i>	Использует левый, правый и двойные повороты для балансировки.	Не требует поворотов.
<i>Структура</i>	Гарантирует, что дерево остается сбалансированным.	Может превращаться в список при неудачном порядке вставки.
<i>Эффективность</i>	Быстрее на больших объемах данных за счет сбалансированной структуры.	Может деградировать до неэффективного случая $O(n)$ при несбалансированности.

Пример 1

Задача:

✘ Реализовать AVL-дерево с поддержкой симметричного обхода.

Решение:

Баланс всех узлов был установлен в 0 просто как заглушка, поскольку балансировка не выполняется.

```

1  #include <iostream>
2  using namespace std;
3
4  // Структура узла AVL-дерева
5  struct Node {
6      int key;           // Значение узла
7      Node* left;        // Левый потомок
8      Node* right;       // Правый потомок
9      int balance;       // Баланс узла
10 };
11
12 // Функция создания нового узла с заданным балансом
13 Node* createNode(int key, int balance, Node* left = nullptr, Node* right = nullptr) {
14     Node* node = new Node;
15     node->key = key;
16     node->left = left;
17     node->right = right;
18     node->balance = balance;
19     return node;
20 }
21
22 // Функция симметричного (in-order) обхода дерева
23 void inOrder(Node* root) {
24     if (root) {
25         inOrder(root->left);
26         cout << root->key << "(" << root->balance << ") ";
27         inOrder(root->right);
28     }
29 }
30
31 int main() {
32     // Изначально заданное AVL-дерево
33     Node* root = createNode(30, 0,
34                             createNode(20, 0, createNode(10, 0), createNode(25, 0)),
35                             createNode(40, 0, nullptr, createNode(50, 0))
36     );
37
38     // Вывод элементов в отсортированном порядке с балансом узлов
39     cout << "Отсортированные элементы с балансом: ";
40     inOrder(root);
41     cout << endl;
42
43     return 0;
44 }

```

Ответ:

Отсортированные элементы с балансом: 10(0) 20(0) 25(0) 30(0) 40(0) 50(0)

Задание 1 (1 балл)

Задача:

Создать двоичное дерево поиска (в узлах хранятся целые положительные числа). Рассчитать для каждого узла построенного дерева показатель баланса

Решение:

Ответ:

Задание 2 (2 балла)
Задача:
 Создать АВЛ-дерево (узлы добавляются как при стандартном алгоритме вставки узла, при нарушении баланса производится необходимая перестройка).
Решение:

Ответ:

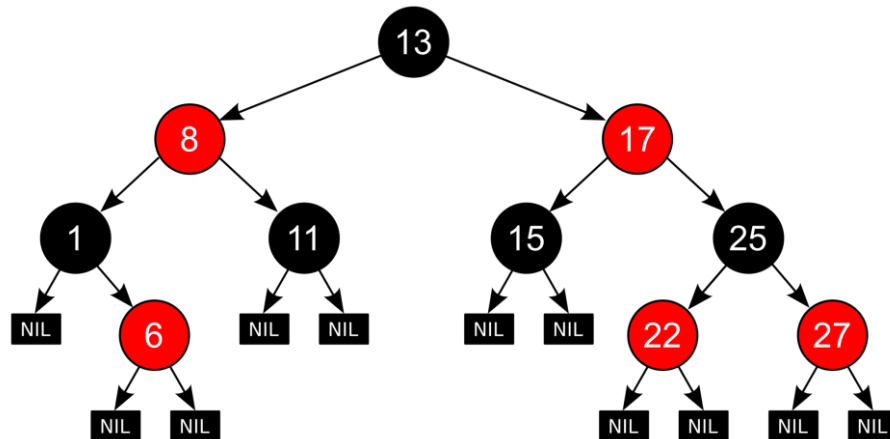
Задание 3*
Задача:
 Для АВЛ-дерева (построенного в задании 2) реализовать операции удаления узла дерева с учетом требования сохранения сбалансированности дерева после удаления узла.
Решение:

Ответ:


2. Красно-черные деревья

Теоретический материал
Красно-черное дерево (англ. red-black tree, RB tree) — один из видов самобалансирующихся двоичных деревьев поиска, гарантирующих логарифмический рост высоты дерева от числа узлов и позволяющее быстро выполнять основные операции дерева поиска: добавление, удаление и поиск узла. Сбалансированность достигается за счёт введения дополнительного атрибута узла дерева — «цвета». Этот атрибут может принимать одно из двух возможных значений — «чёрный» или «красный». Свойства КЧ-деревьев:

1. каждый узел либо красный, либо черный;
2. каждый лист (фиктивный) – черный;
3. если узел красный, то оба его сына – черные;
4. все пути, идущие от корня к любому фиктивному листу, содержат одинаковое количество черных узлов;
5. корень – черный.



Структура данных, описывающая узел красно-черного дерева, имеет вид:

```

Struct Node{
int key;
Node * left;
Node * right;
Node * parent;
char color;
};
  
```

Для упрощения работы алгоритмов в структуру допускается ввести указатель на родительский узел.

Задание 4 (2 балла)

Задача:

Создать двоичное дерево поиска (в узлах хранятся целые положительные числа). Проверить для построенного дерева свойства красно-черного дерева.

Решение:



Ответ:



Задание 5*	
Задача:	
	Создать красно-черное дерево (узлы добавляются как при стандартном алгоритме вставки узла, при нарушении свойств красно-черного дерева производится необходимая перестройка и перекрашивание). Реализовать операцию удаления узла из красно-черного дерева.
Решение:	
	
Ответ:	
	

3. Самоперестраивающиеся деревья (splay деревья)

Теоретический материал	
Самоперестраивающееся дерево является двоичным деревом поиска, в котором поддерживается свойство сбалансированности. Это дерево принадлежит классу «саморегулирующихся деревьев», которые поддерживают необходимый баланс ветвления дерева, чтобы обеспечить выполнение операций поиска, добавления и удаления за логарифмическое время от числа хранимых элементов. Это реализуется без использования каких-либо дополнительных полей в узлах дерева (как, например, в Красно-чёрных деревьях или АВЛ-деревьях, где в вершинах хранится, соответственно, цвет вершины и глубина поддеревя). Вместо этого «расширяющие операции» (splay operation), частью которых являются вращения, выполняются при каждом обращении к дереву. Операция $splay(T, k)$ выполняется следующим образом. Сначала производится поиск узла с ключом k в дереве обычным способом, спускаясь вниз, начиная с корня. При этом запоминается пройденный путь. В итоге, получаем указатель на узел дерева либо с ключом k, либо с его предшественником или последователем, на котором закончился поиск. Далее, происходит возвращение назад по запомненному пути, с перемещением этого узла к корню. Для того, чтобы при этом сохранялись свойства двоичного дерева поиска, необходимы повороты.	
Задание 6*	

Задача:

Создать самоперестраивающееся дерево, для которого добавление нового узла и удаление существующего узла будет производиться на основе операции splay.

Решение:**Ответ:**